

Datenbank-Analytik mit SQL

Prof. Dr. Michael Kaufmann

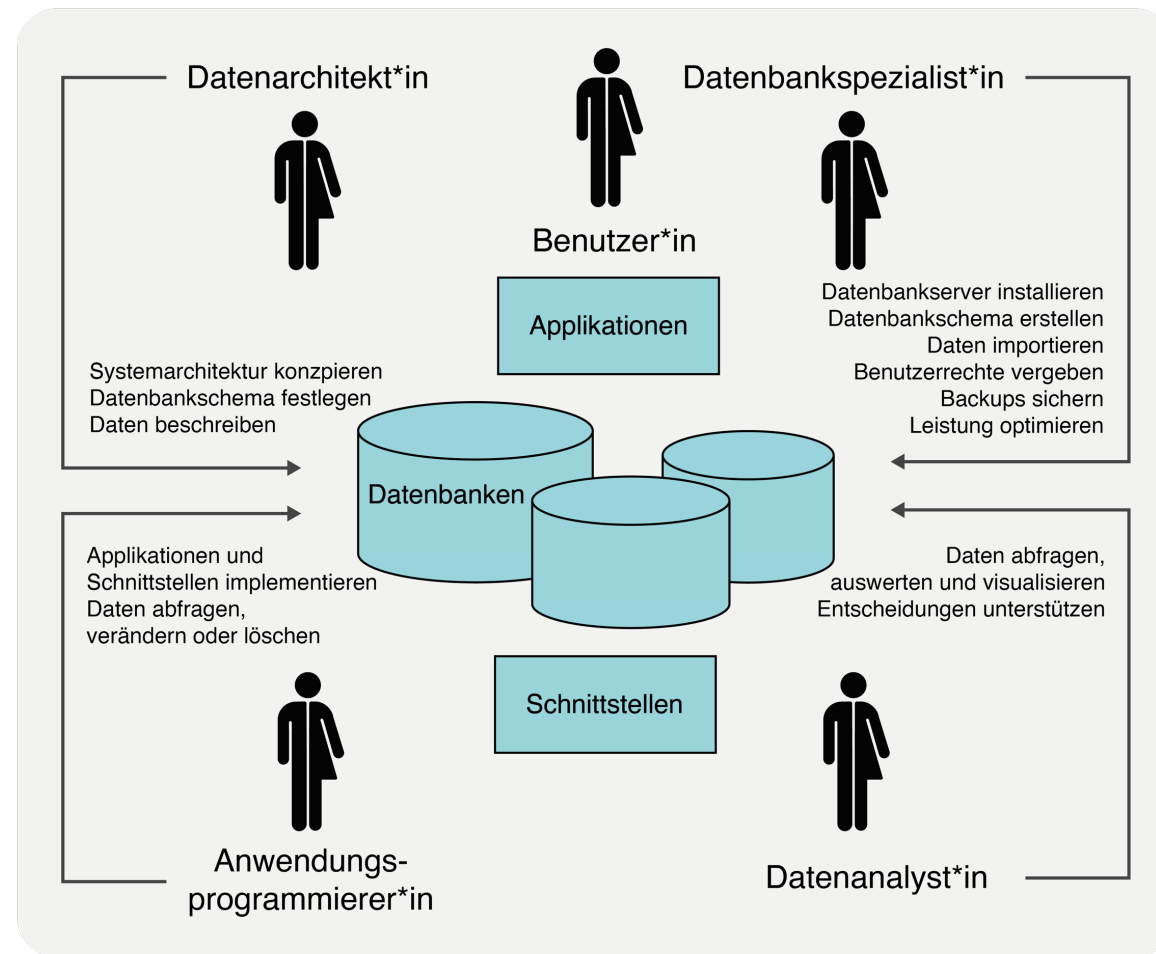
Informatik
21. Oktober 2025

FH Zentralschweiz



BACKGROUND OF SQL

Interaction with a database through a language API



Relational database languages: Historical root – SEQUEL

Chamberlin, D. D., Boyce, R. F. (1974). *SEQUEL: A structured English query language*. SIGFIDET '74 Proceedings of the 1974 ACM SIGFIDET (now SIGMOD) workshop on Data description, access and control



- SEQUEL ("Structured English QUery Language") was created for "System R" in the mid-seventies. As a test system, it was one of the first executable relational database systems.
- SEQUEL was a relationally complete query language not based on mathematical symbols but on *English words* like 'select', 'from', 'where', 'count', 'group by', etc.



Relational database language SQL

- In 1979, Relational Software, Inc. (known today as Oracle Corporation) introduced a new version of SEQUEL in a software named Oracle V2, for VAX computers to the market. The language was called SQL (Structured Query Language)
- Because of its historical roots, some people today still pronounce the abbreviation "S.Q.L." as "*Sequel*."
- ANSI, and later ISO, normed and further developed SQL at a later stage: SQL-86, SQL-89, SQL-92, SQL:1999, SQL:2003, SQL:2006, SQL:2008, SQL:2011, SQL:2016, ...
- SQL is as powerful as relational algebra and is therefore considered a *relationally complete* language.
- SQL has been the leading language for database interactions for decades. Under NewSQL, many NoSQL ("not only SQL") systems offer an SQL language interface.

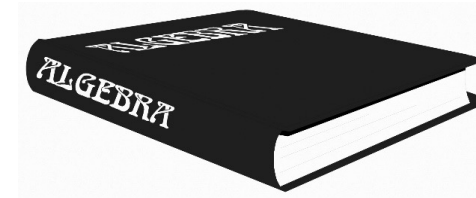
Interaktion

- SQL is 50 years old. (1974)
- Why do you think this “prehistoric” language is still (or again) so widespread today?



RELATIONAL OPERATIONS OF SQL

Relationally complete database languages



SQL is as powerful as *relational algebra* and is therefore considered a *relationally complete* language.

The relational completeness of a database language means that the operators of relational algebra (see Chapter 3.2) can be represented in it:

- **Selection** (choice of rows)
- **Projection** (choice of columns)
- **Cartesian product** (set of all possible combinations of rows of two tables)
- Plus the set-based operations **union**
- and **difference**

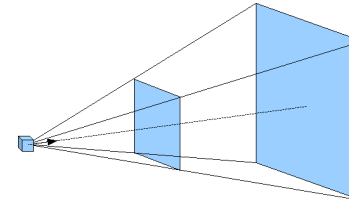
$$\begin{array}{ccc} F(A) & \xrightarrow{\alpha} & A \\ F(f) \downarrow & & \downarrow f \\ F(B) & \xrightarrow{\beta} & B \end{array}$$

- (... and, just in case columns have no order:
- additional renaming due to elimination
- of duplicate names)



SELECT: Projection

```
SELECT Sub, Name  
FROM Employee
```



Since SQL, unlike relational algebra, does not eliminate duplicates, the word **DISTINCT** must be included in the **SELECT** clause:

```
SELECT DISTINCT City  
FROM Employee;
```

EMPLOYEE

E#	Name	Street	City	Sub
E19	Stewart	E Main Street	Stow	D6
E1	Murphy	Murray Road	Kent	D3
E7	Howard	Lorain Avenue	Cleveland	D5
E4	Bell	S Water Street	Kent	D6

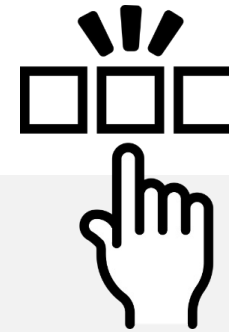
$\pi_{\text{City}}(\text{EMPLOYEE})$

City
Stow
Kent
Cleveland

$\pi_{\text{Sub, Name}}(\text{EMPLOYEE})$

Sub	Name
D6	Stewart
D3	Murphy
D5	Howard
D6	Bell

WHERE: Selection



```
SELECT    *  
FROM      EMPLOYEE  
WHERE     City = 'Kent' AND Sub='D6'
```

- An asterisk (*) in the SELECT clause means that all attributes of the corresponding tables are selected; the results table thus contains the attributes E#, name, street, city, and "Sub" (subordination).
- The WHERE clause contains the desired selection predicate. Possible logical operators are AND, OR, NOT
- Executing the above query by the database system thus leads to employee Becker from Liestal, who works in department A6.

$\sigma_{\text{City=Kent AND Sub=D6}}(\text{EMPLOYEE})$

E#	Name	Street	City	Sub
E4	Bell	S Water Street	Kent	D6

FROM: Table selection (Cartesian product)

```
SELECT      E#, Name, Steet, Place, Sub, D#, DepartmentName
FROM        EMPLOYEE, DEPARTMENT
```

EMPLOYEE

E#	Name	Street	City	Sub
E19	Stewart	E Main Street	Stow	D6
E1	Murphy	Murray Road	Kent	D3
E7	Howard	Lorain Avenue	Cleveland	D5
E4	Bell	S Water Street	Kent	D6

DEPARTMENT

D#	DepartmentName
D3	IT
D5	HR
D6	Accounting



EMPLOYEE x DEPARTMENT

E#	Name	Street	City	Sub	D#	DepartmentName
E19	Stewart	E Main Street	Stow	D6	D3	IT
E19	Stewart	E Main Street	Stow	D6	D5	HR
E19	Stewart	E Main Street	Stow	D6	D6	Accounting
E1	Murphy	Murray Road	Kent	D3	D3	IT
E1	Murphy	Murray Road	Kent	D3	D5	HR
E1	Murphy	Murray Road	Kent	D3	D6	Accounting
E7	Howard	Lorain Avenue	Cleveland	D5	D3	IT
E7	Howard	Lorain Avenue	Cleveland	D5	D5	HR
E7	Howard	Lorain Avenue	Cleveland	D5	D6	Accounting
E4	Bell	S Water Street	Kent	D6	D3	IT
E4	Bell	S Water Street	Kent	D6	D5	HR
E4	Bell	S Water Street	Kent	D6	D6	Accounting

FROM + WHERE: Inner join

```
SELECT  E#, Name, Street, City, Sub, D#, Sub
FROM    EMPLOYEE, DEPARTMENT
WHERE   Sub=D#
```

EMPLOYEE

E#	Name	Street	City	Sub
E19	Stewart	E Main Street	Stow	D6
E1	Murphy	Murray Road	Kent	D3
E7	Howard	Lorain Avenue	Cleveland	D5
E4	Bell	S Water Street	Kent	D6

DEPARTMENT

D#	DepartmentName
D3	IT
D5	HR
D6	Accounting

EMPLOYEE |x|_{Sub=D#} DEPARTMENT

E#	Name	Street	City	Sub	D#	DepartmentName
E19	Stewart	E Main Street	Stow	D6	D6	Accounting
E1	Murphy	Murray Road	Kent	D3	D3	IT
E7	Howard	Lorain Avenue	Cleveland	D5	D5	HR
E4	Bell	S Water Street	Kent	D6	D6	Accounting

- 
- Which employee works in which department?

Renaming (aliases in SQL)

```
SELECT
    S1.Name as Participant1,
    S2.Name as Participant2
FROM SPORTS_CLUB as S1, SPORTS_CLUB as S2
WHERE S1.E# <> S2.E#
```

13, 21.10.25

SPORTS_CLUB

E#	Name	Street	City
E1	Murphy	Murray Road	Kent
E7	Howard	Lorain Avenue	Cleveland
E19	Stewart	E Main Street	Stow



SPORT COMPETITION

Participant1	Participant2
Murphy	Howard
Murphy	Stewart
Howard	Murphy
Howard	Stewart
Stewart	Murphy
Stewart	Howard

*If identical column names occur
in the same query =>
Differentiation with alias*

SET-ORIENTED OPERATORS OF SQL

Set Union

```
SELECT      *
FROM        SPORTS_CLUB
UNION
SELECT      *
FROM        PHOTO_CLUB
```

*The tables to be unified must be
UNION-compatible => same
number of columns with
compatible data types*

SPORTS_CLUB

E#	Name	Street	City
E1	Murphy	Murray Road	Kent
E7	Howard	Lorain Avenue	Cleveland
E19	Stewart	E Main Street	Stow

PHOTO_CLUB

E#	Name	Street	City
E4	Bell	S Water Street	Kent
E7	Howard	Lorain Avenue	Cleveland

CLUB_MEMBERS = SPORTS_CLUB $\cup$ PHOTO_CLUB

E#	Name	Street	City
E1	Murphy	Murray Road	Kent
E7	Howard	Lorain Avenue	Cleveland
E19	Stewart	E Main Street	Stow
E4	Bell	S Water Street	Kent

Intersection

```
SELECT *  
FROM SPORTS_CLUB  
WHERE SPORTS_CLUB.E# IN (  
    SELECT E# FROM PHOTO_CLUB  
)
```

SPORTS_CLUB

E#	Name	Street	City
E1	Murphy	Murray Road	Kent
E7	Howard	Lorain Avenue	Cleveland
E19	Stewart	E Main Street	Stow

PHOTO_CLUB

E#	Name	Street	City
E4	Bell	S Water Street	Kent
E7	Howard	Lorain Avenue	Cleveland



E#	Name	Street	City
E7	Howard	Lorain Avenue	Cleveland

Set Difference

```
SELECT      *
FROM        SPORTS_CLUB, PHOTO_CLUB
WHERE       SPORTS_CLUB.E# NOT IN (
            SELECT E# FROM PHOTO_CLUB)
```

SPORTS_CLUB

E#	Name	Street	City
E1	Murphy	Murray Road	Kent
E7	Howard	Lorain Avenue	Cleveland
E19	Stewart	E Main Street	Stow

PHOTO_CLUB

E#	Name	Street	City
E4	Bell	S Water Street	Kent
E7	Howard	Lorain Avenue	Cleveland

COMPETITION = (SPORTS_CLUB \ PHOTO_CLUB) × PHOTO_CLUB

E#	Name	Street	City	E#	Name	Street	City
E1	Murphy	Murray Road	Kent	E4	Bell	S Water Street	Kent
E1	Murphy	Murray Road	Kent	E7	Howard	Lorain Avenue	Cleveland
E19	Stewart	E Main Street	Stow	E4	Bell	S Water Street	Kent
E19	Stewart	E Main Street	Stow	E7	Howard	Lorain Avenue	Cleveland

NULL VALUES

Überraschende Ergebnisse beim Vereinigen mit Nullwerten

MITARBEITENDE

M#	Name	Straße	Ort	Unt
M19	Schweizer	Hauptstraße	Frenkendorf	A6
M1	Meier	?	?	A3
M7	Huber	Mattenweg	Basel	A5
M4	Becker	?	?	A6

Was ist hier die Inkonsistenz?

```
SELECT *  
FROM   MITARBEITER  
WHERE  Ort = 'Liestal'  
UNION  
SELECT *  
FROM   MITARBEITER  
WHERE  NOT Ort = 'Liestal'
```

RESULTATTABELLE

M#	Name	Straße	Ort	Unt
M19	Schweizer	Hauptstraße	Frenkendorf	A6
M7	Huber	Mattenweg	Basel	A5

Wie kommt es dazu?

Nullwerte: Dreiwertige Logik

Ein Nullwert steht für einen Datenwert einer Tabellenspalte, der (noch) nicht bekannt ist. Dies entspricht einem dritten **Wahrheitswert ?** (unbekannt), der zusätzlich zu den geläufigen Wahrheitswerten **0** (falsch) **1** (wahr) und bei Datenbanken dazukommt.

Logische Ausdrücke werden nach den folgenden **Wahrheitstabellen** berechnet:

OR	1	?	0
1	1	1	1
?	1	?	?
0	1	?	0

AND	1	?	0
1	1	?	0
?	?	?	0
0	0	0	0

NOT	
1	0
?	?
0	1

Selektion über Nullwerte

- `insert into Studenten (MatrNr, Name)
values (12345, 'Edison')`
- `select * from Studenten where Semester is null`
- `select * from Studenten where Semester is not null`

Result Grid			Filter Rows:	Search
MatrNr	Name	Semester		
12345	Edison	NULL		

Result Grid			Filter Rows:	Search
MatrNr	Name	Semester		
24002	Xenokrates	19		
25403	Jonas	12		
26120	Fichte	10		
26830	Aristoxenos	8		
27550	Schopenhauer	6		

DATENANALYSEN MIT SQL

Aggregatfunktionen



Was ist das
Resultat dieser
Query?

```
SELECT    COUNT (M#)
FROM      MITARBEITER
WHERE     Unt='A6'
```

MITARBEITER

M#	Name	Straße	Ort	Unt
M19	Schweizer	Hauptstraße	Frenkendorf	A6
M1	Meier	Lindenstraße	Liestal	A3
M7	Huber	Mattenweg	Basel	A5
M4	Becker	Wasserweg	Liestal	A6

Aggregatfunktionen
berechnen ausgehend von einer
Menge von Werten einen
einzigsten skalaren Wert:

- COUNT(X)
Anzahl Werte
- COUNT(DISTINCT X)
Zählung ohne Duplikate
- SUM: Summenbildung
- AVG: Mittelwert
- MAX: Maximalwert
- MIN: Minimalwert

Gruppierung von Aggregaten



Was ist das
Resultat dieser
Query?

```
SELECT    COUNT (M#),  Unt
FROM      MITARBEITER
GROUP BY  Unt
```

MITARBEITER

M#	Name	Straße	Ort	Unt
M19	Schweizer	Hauptstraße	Frenkendorf	A6
M1	Meier	Lindenstraße	Liestal	A3
M7	Huber	Mattenweg	Basel	A5
M4	Becker	Wasserweg	Liestal	A6

Aggregatfunktionen mit
Gruppierung
berechnen ausgehend von einer
Menge von Werten einen
skalaren Wert, und zwar für jede
Ausprägung, die in der
Spalte(nkombination) in der
GROUP BY Klausel vorkommt

Descriptive statistics with SQL: Mean value and standard deviation

- 1.) Connect dimension and indicator tables (JOIN)
 - 2.) Aggregate indicator attributes (SUM / AVG / STDDEV / MIN / MAX)
 - 3.) Group by dimension attributes (GROUP BY) and select (WHERE)
- E.g. mean and standard deviance of turnover per supplier

```
SELECT      s.supplier_name, AVG(u.turnover_chf),  
            STDDEV(u.turnover_chf)  
  
FROM        ind_turnover u  
  
JOIN        dim_supplier s USING(supplier_key)  
  
JOIN        dim_time t USING(time_key)  
  
WHERE       t.yearmonth = 201901  
            AND s.category = 'Software'  
  
GROUP BY    s.supplier_name
```

Dimension

Indicators
(aggregate)

Selection

grouping

Bayesian statistics with SQL: Probabilities and frequency distributions

Probability: $p(x) = |\{t \in T \mid X_t = x\}| / |T|$

$|T|$ can be calculated with a windowing function

Example: Probability that an order will be delivered by a certain supplier x , as a time series over quarters q

```
SELECT      s.supplier_name as x,  
             t.quarter as q,  
             COUNT(*) as n, COUNT(*) / COUNT(*) OVER() as P  
  
FROM        ind_orders o  
  
JOIN        dim_supplier s USING (supplier_key)  
  
JOIN        dim_time t USING (time_key)  
  
GROUP BY    s.supplier_name, t.quarter
```

Variable (X)

Empirical
probability of x



Unterabfrage

Was ist die durchschnittliche
Anzahl Lektionen pro Woche,
Die ein Professor hat?
(Avg. Summe der SWS pro Prof.) ?



Wie könnte
man dieses
Problem lösen?

```
1 select avg(sum(sws))
2 from professoren, vorlesungen
3 where gelesenVon = persNr
4 group by name
5
```

100% 8:1

Action Output

	Time	Action	Response
✖ 1	11:14:33	select avg(sum(sws)) from professoren, vor...	Error Code: 1111. Invalid use of group function

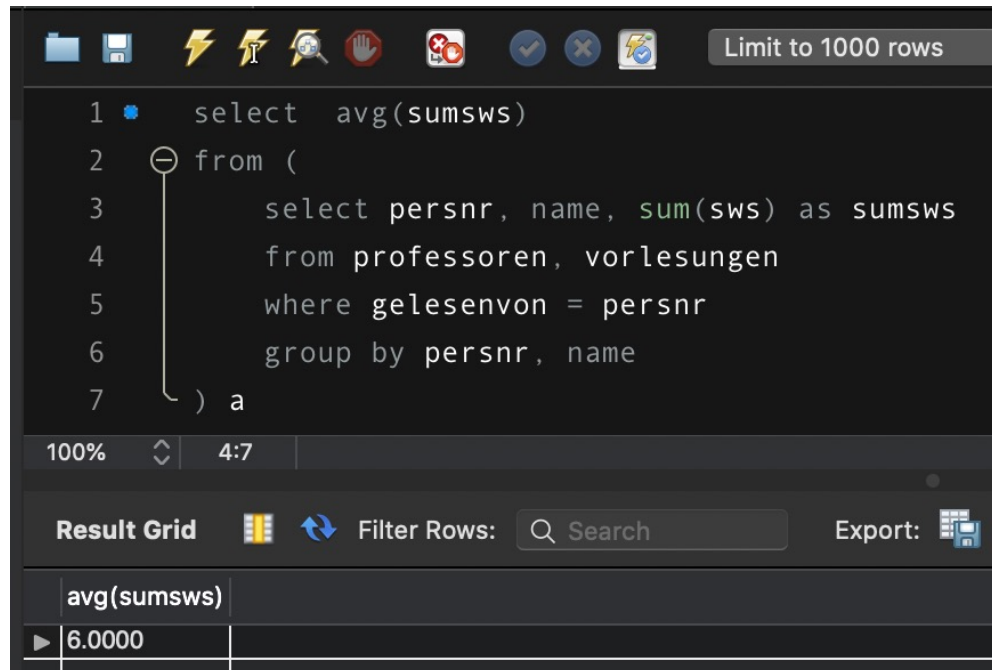
Unterabfrage

```
SELECT ...  
FROM (  
    SELECT...  
    FROM...  
    WHERE ...
```

```
) alias
```

```
WHERE ...
```

Was ist die durchschnittliche
Anzahl Lektionen
pro Woche,
Die ein Professor hat?
(Avg. Summe der SWS) ?



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' button. The SQL editor contains the following query:

```
1 select avg(sumsws)  
2 from (  
3     select persnr, name, sum(sws) as sumsws  
4     from professoren, vorlesungen  
5     where gelesenvon = persnr  
6     group by persnr, name  
7 ) a
```

Below the editor, the 'Result Grid' is displayed with the following data:

avg(sumsws)
6.0000



EXAMPLE: MOVIE RECOMMENDER

Reminder: Filmempfehlungssystem

Datennutzung

- ▶ Anwendungsfall: Filmempfehlung
- ▶ Entscheidungshilfe: Welchen Film schauen?
- ▶ Datenquellen: Filme, Bewertungen (MovieLens)
- ▶ Datenvisualisierung: BI-Tool (Metabase)
- ▶ Interaktion mit dem Benutzer: Eingabe Film, Ausgabe Empfehlungen

Datentechnik

- ▶ Datenbank-Software: MySQL Server, MySQL Workbench
- ▶ Integriertes System: Metabase mit SQL-API
- ▶ Dateneingabe: MySQL LOAD
- ▶ Datenverarbeitung: SQL SELECT queries, Transformation mit INSERT INTO ... SELECT
- ▶ Leistung: Materialisierung: CREATE TABLE AS SELECT; Indexierung: CREATE INDEX

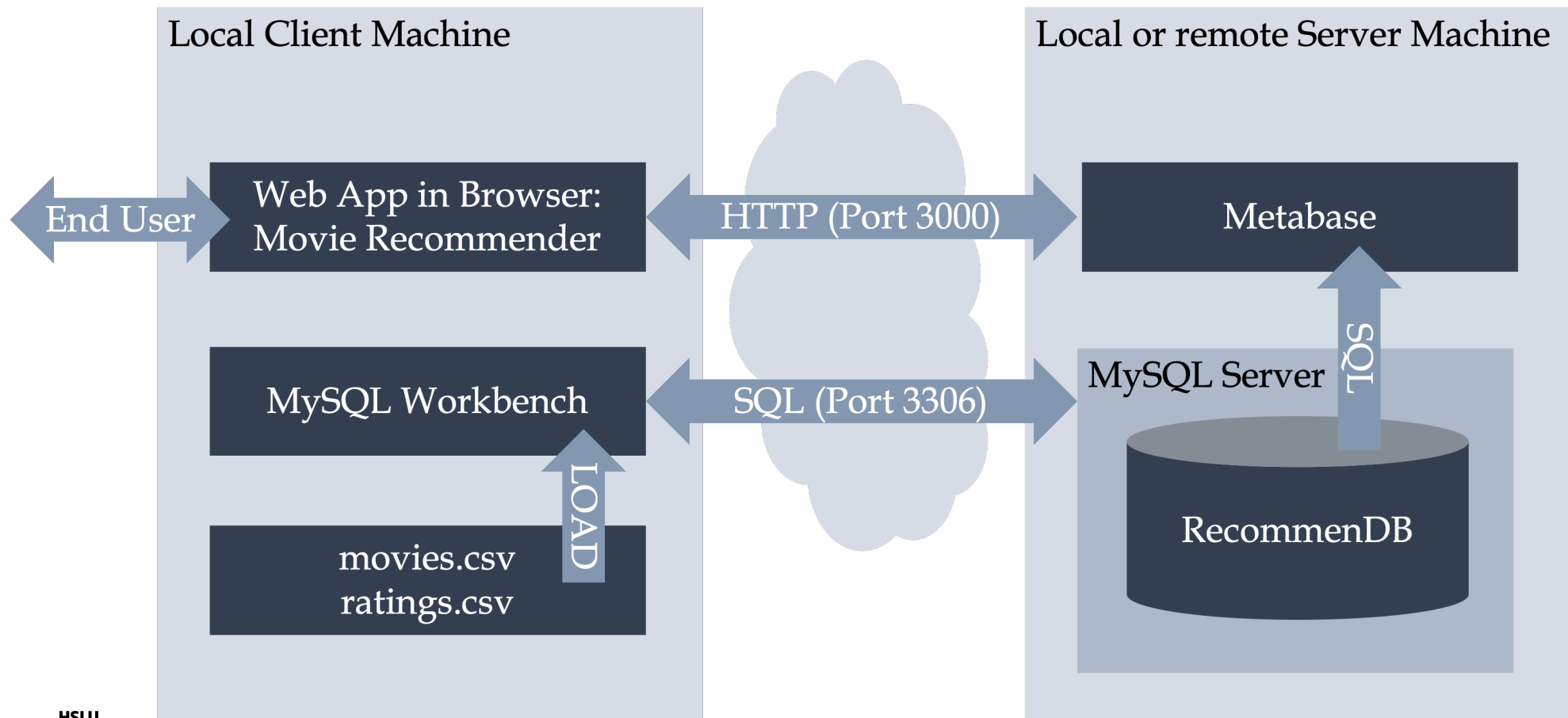
Datenadministration

- ▶ Operative Organisation: Entwickler, End User
- ▶ Datenzugang:
 - ▶ Entwickler über MySQL Workbench,
 - ▶ End User über Metabase
- ▶ Zugriffsrechte:
 - ▶ Admin für Entwickler via Port 3306
 - ▶ Eingeschränkte Leserechte für Metabase

Datenarchitektur

- ▶ Systemkomponenten (siehe nächste Folie)
- ▶ Datenmodellierung:
 - ▶ Filme mit Titel, Genres, Jahr;
 - ▶ Bewertungen mit Benutzer, Film, Zeitstempel, Bewertung von 1-5 Sternen
- ▶ Datenbank-Schema: SQL CREATE TABLE

Reminder: Systemarchitektur



Reminder: Source Data

PowerShell

PowerShell 7.3.6
PS C:\Users\micha> get-content C:\Users\micha\Downloads\ml-25m\ml-25m\ratings.csv | select -First 15
userId,movieId,rating,timestamp
1,296,5.0,1147880044
1,306,3.5,1147868817
1,307,5.0,1147868828
1,665,5.0,1147878820
1,899,3.5,1147868510
1,1088,4.0,1147868495
1,1175,3.5,1147868826
1,1217,3.5,1147878326
1,1237,5.0,1147868839
1,1250,4.0,1147868414
1,1260,3.5,1147877857
1,1653,4.0,1147868097
1,2011,2.5,1147868079
1,2012,2.5,1147868068
PS C:\Users\micha>

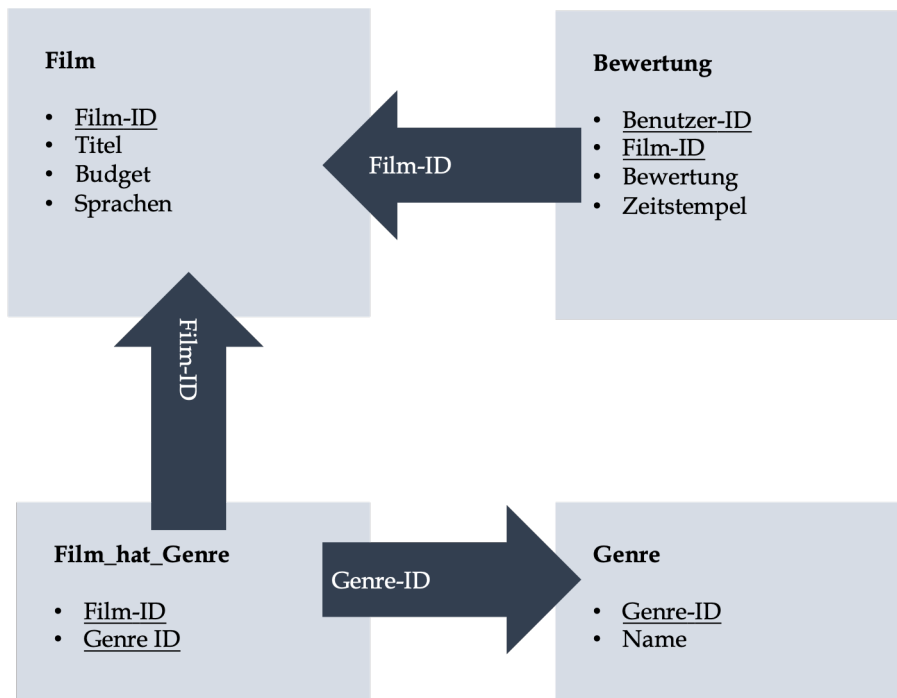
movies.csv - Editor

Datei Bearbeiten Ansicht

"MovieID"	"Title"	"Release_date"	"Budget"	"Genres"	"Spoken_languages"
"1"	"Toy Story"	"1995-10-30"	"30000000"	"[{"id": 16, "name": "Animation"}, {"id": 35,	
"2"	"Jumanji"	"1995-12-15"	"65000000"	"[{"id": 12, "name": "Adventure"}, {"id": 14,	
"3"	"Grumpier Old Men"	"1995-12-22"	"0"	"[{"id": 10749, "name": "Romance"}, {"id	
"4"	"Waiting to Exhale"	"1995-12-22"	"16000000"	"[{"id": 35, "name": "Comedy"}, {"	
"5"	"Father of the Bride Part II"	"1995-02-10"	"0"	"[{"id": 35, "name": "Comedy"}]"	
"6"	"Heat"	"1995-12-15"	"60000000"	"[{"id": 28, "name": "Action"}, {"id": 80, \"	
"7"	"Sabrina"	"1995-12-15"	"58000000"	"[{"id": 35, "name": "Comedy"}, {"id": 10749,	
"8"	"Tom and Huck"	"1995-12-22"	"0"	"[{"id": 28, "name": "Action"}, {"id": 12, \"	
"9"	"Sudden Death"	"1995-12-22"	"35000000"	"[{"id": 28, "name": "Action"}, {"id":	
"10"	"GoldenEye"	"1995-11-16"	"58000000"	"[{"id": 12, "name": "Adventure"}, {"id": 28,	
"11"	"The American President"	"1995-11-17"	"62000000"	"[{"id": 35, "name": "Comedy\"	
"12"	"Dracula: Dead and Loving It"	"1995-12-22"	"0"	"[{"id": 35, "name": "Comedy"}, {"	
"13"	"Balto"	"1995-12-22"	"0"	"[{"id": 10751, "name": "Family"}, {"id": 16, \"nam	
"14"	"Nixon"	"1995-12-22"	"44000000"	"[{"id": 36, "name": "History"}, {"id": 18, \"	
"15"	"Cutthroat Island"	"1995-12-22"	"98000000"	"[{"id": 28, "name": "Action"}, {"	
"16"	"Casino"	"1995-11-22"	"52000000"	"[{"id": 18, "name": "Drama"}, {"id": 80, \"n	
"17"	"Sense and Sensibility"	"1995-12-13"	"16500000"	"[{"id": 18, "name": "Drama"}, {"i	

Ze 6, Sp 31100%Unix (LF)UTF-8

Reminder: Normalisiertes Datenmodell und DB-Schema für Filmempfehlungssystem



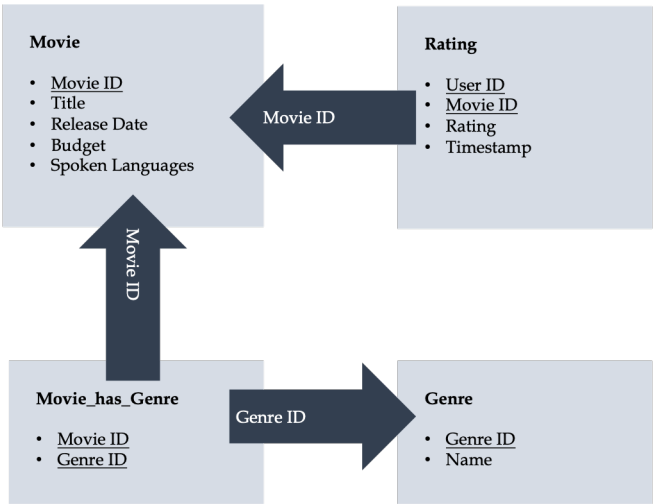
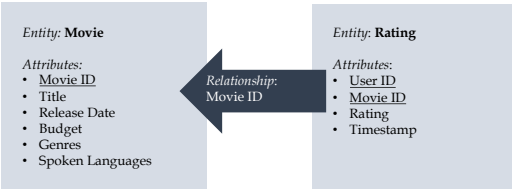
```
CREATE TABLE Movies (  
  MovieID INTEGER PRIMARY KEY,  
  Title TEXT,  
  Release_date DATE,  
  Budget INTEGER,  
  Spoken_languages JSON );
```

```
CREATE TABLE Ratings (  
  UserID INTEGER,  
  MovieID INTEGER,  
  Rating INTEGER,  
  Timestamp DATETIME,  
  PRIMARY KEY (MovieID, UserID),  
  CONSTRAINT MovieID_FK  
  FOREIGN KEY (MovieID)  
  REFERENCES Movies (MovieID));
```

```
CREATE TABLE Movie_has_Genre (  
  MovieID INTEGER,  
  GenreID INTEGER,  
  PRIMARY KEY (MovieID, GenreID),  
  FOREIGN KEY (MovieID)  
  REFERENCES Movies (MovieID),  
  FOREIGN KEY (GenreID)  
  REFERENCES genres (GenreID));
```

```
CREATE TABLE Genres (  
  GenreID INTEGER PRIMARY KEY,  
  Name TEXT);
```

Reminder: Laden & Transformation



```
PowerShell 7.3.6
PS C:\Users\micha> get-content C:\Users\micha\Downloads\ml-25m\ml-25m\ratings.csv | select -First 15
userId,movieId,rating,timestamp
1,296,5.0,1147880044
1,306,3.5,1147868817
1,307,5.0,1147868828
1,665,5.0,1147878820
1,899,3.5,1147868510
1,1088,4.0,1147868495
1,1175,3.5,1147868826
1,1217,3.5,1147878326
1,1237,5.0,1147868839
1,1250,4.0,1147868414
1,1260,3.5,1147877857
1,1653,4.0,1147868097
1,2011,2.5,1147868079
1,2012,2.5,1147868068
PS C:\Users\micha>
```

movies.csv - Editor

Datei

Bearbeiten

Ansicht

Load & Trans

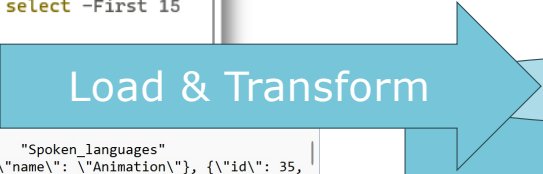
"MovieID"	"Title"	"Release_date"	"Budget"	"Genres"	"Spoken_languages"
"1"	"Toy Story"	"1995-10-30"	"300000000"	"[{\"id\": 16, \"name\": \"Animation\"}, {\"id\": 35,	
"2"	"Jumanji"	"1995-12-15"	"650000000"	"[{\"id\": 12, \"name\": \"Adventure\"}, {\"id\": 14,	
"3"	"Grumpier Old Men"	"1995-12-22"	"0"	"[{\"id\": 10749, \"name\": \"Romance\"}, {\"id	
"4"	"Waiting to Exhale"	"1995-12-22"	"160000000"	"[{\"id\": 35, \"name\": \"Comedy\"}, {\"id	
"5"	"Father of the Bride Part II"	"1995-02-10"	"0"	"[{\"id\": 35, \"name\": \"Comedy\"}]"	
"6"	"Heat"	"1995-12-15"	"600000000"	"[{\"id\": 28, \"name\": \"Action\"}, {\"id\": 80, \"	
"7"	"Sabrina"	"1995-12-15"	"580000000"	"[{\"id\": 35, \"name\": \"Comedy\"}, {\"id\": 10749,	
"8"	"Tom and Huck"	"1995-12-22"	"0"	"[{\"id\": 28, \"name\": \"Action\"}, {\"id\": 12, \"	
"9"	"Sudden Death"	"1995-12-22"	"350000000"	"[{\"id\": 28, \"name\": \"Action\"}, {\"id\":	
"10"	"GoldenEye"	"1995-11-16"	"580000000"	"[{\"id\": 12, \"name\": \"Adventure\"}, {\"id\": 28,	
"11"	"The American President"	"1995-11-17"	"620000000"	"[{\"id\": 35, \"name\": \"Comedy\"}	
"12"	"Dracula: Dead and Loving It"	"1995-12-22"	"0"	"[{\"id\": 35, \"name\": \"Comedy\"}, {\"	
"13"	"Balto"	"1995-12-22"	"0"	"[{\"id\": 10751, \"name\": \"Family\"}, {\"id\": 16, \"nam	
"14"	"Nixon"	"1995-12-22"	"440000000"	"[{\"id\": 36, \"name\": \"History\"}, {\"id\": 18, \"	
"15"	"Cutthroat Island"	"1995-12-22"	"980000000"	"[{\"id\": 28, \"name\": \"Action\"}, {\"	
"16"	"Casino"	"1995-11-22"	"520000000"	"[{\"id\": 18, \"name\": \"Drama\"}, {\"id\": 80, \"n	
"17"	"Sense and Sensibility"	"1995-12-13"	"165000000"	"[{\"id\": 18, \"name\": \"Drama\"}, {\"i	

Ze 6, Sp 31

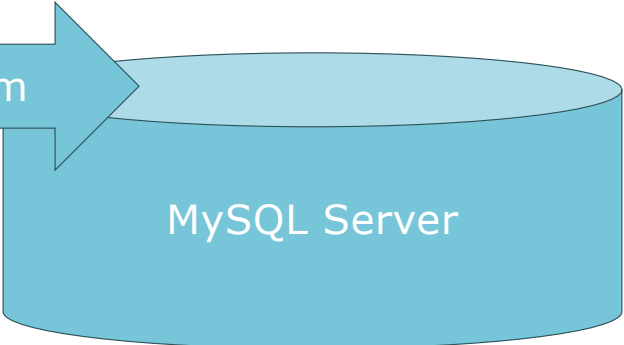
100%

Unix (LF)

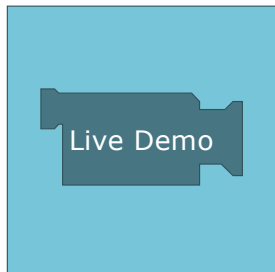
UTF-8



Normalized DB-Schema



SQL: Views



Navigator

SCHEMAS

Filter objects

- ▼ recommenddb
 - ▼ Tables
 - movies
 - ratings
 - ▼ Views
 - ratings5
 - Stored Procedures
 - Functions
- ▼ sys

Query 1 x

Limit to 1000

```
1 • CREATE OR REPLACE VIEW Ratings5 AS
2   SELECT UserID, m.MovieID, Title
3   FROM Ratings r, Movies m
4   WHERE rating = 5
5   AND r.MovieID = m.MovieID;
6
7 • SELECT * from Ratings5;
```

Result Grid

	UserID	MovieID	Title
▶	58365	32	Twelve Monkeys
	58365	62	Mr. Holland's Opus
	58365	110	Braveheart
	58365	141	The Birdcage
	58365	150	Apollo 13
	58365	161	Crimson Tide
	58365	173	Judge Dredd

Film-Empfehlung mit SQL

Was sind die Stärken und Schwächen dieser Art von Filmempfehlung mit SQL?



```
SELECT MovieID, Title,
       COUNT(*) AS N5R
FROM Ratings5
WHERE UserID IN (
  SELECT UserID
  FROM Ratings5
  WHERE MovieID = 1371 )
GROUP BY MovieID
ORDER BY COUNT(*) DESC
LIMIT 10;
```

=> 10 Zeilen Code

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows: |

	MovieID	Title	N5R
▶	1371	Star Trek: The Motion Picture	1256
	260	Star Wars	654
	1196	The Empire Strikes Back	593
	1198	Raiders of the Lost Ark	541
	1210	Return of the Jedi	538
	1374	Star Trek II: The Wrath of Khan	500
	2571	The Matrix	463
	1376	Star Trek IV: The Voyage Home	444
	1375	Star Trek III: The Search for Spock	437
	1356	Star Trek: First Contact	425

Result 12 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 1	20:08:29	SELECT MovieId, Title, C...	1000 row(s) returned	46.562 sec / 0.000 sec

Gehen wir den Code Zeile für Zeile durch.
Was bewirkt jede Zeile dieses Codes?

Erklärung SQL-Code Filmempfehlung

- 1: Spalten auswählen: MovieId, Title
- 2: Anzahl Zeilen in der Unterabfrage, gruppiert nach MovieID & Title
- 3: Tabelle auswählen: Ratings5
- 4 & 5: Unterabfrage: UserIDs
- 6: Tabelle für Unterabfrage: Ratings5 nochmal
- 7: Film auswählen für Unterabfrage
- 8: Gruppieren nach Movie ID
- 9: Sortiert nach Häufigkeit der 5-Sterne-Bewertung der Zielgruppe pro Film
- 10: Gibt nur die ersten 10 Zeilen aus.

Danke!
Fragen?

Hochschule Luzern
Informatik
Forschung
Prof. Dr. Michael Kaufmann
Dozent

T direkt +41 41 757 68 48
m.kaufmann@hslu.ch